Monte Carlo y MC Retest

AlgoProject — manual de uso. Actualizado el 2026-10-12 con `tools/manual.py`.

1. 7. Monte Carlo — cuánto de este resultado es suerte, y de qué tipo
2. 32. Configurar el MC Retest en SQX — las ocho tareas, escritas de una vez
3. 11. Retest de Monte Carlo — ¿habrían ocurrido siquiera estas operaciones?

7. Monte Carlo — cuánto de este resultado es suerte, y de qué tipo

Qué pregunta responde

Una estrategia que llega hasta aquí ya te ha convencido: sobrevivió al decaimiento fuera de muestra y al retest en otros mercados. Este análisis **no vuelve a preguntar si tiene edge**. Pregunta de qué depende ese edge, que es otra cosa:

- **¿Del orden en que llegaron las operaciones?** Las mismas operaciones barajadas dan el mismo beneficio pero un drawdown distinto. Si el drawdown real fue mucho menor que el de las barajadas, tuviste suerte con el orden y tu cuenta no está dimensionada para lo que viene.
- **¿De unas pocas operaciones concretas?** Se vuelven a sortear con reemplazo. Si en 1 de cada 20 sorteos el resultado es negativo, el beneficio dependía de que salieran justo ésas.
- **¿De que la ejecución fuera buena?** Se repite todo con costes hasta el doble, spread más ancho, entradas perdidas y ejecuciones peores.
- **¿De un régimen de mercado que ya no está?** Se parte la historia en ventanas de dos años y en terciles de volatilidad diaria, y se mira si ganaba en todos o solo en uno.

Las cuatro familias, además, responden por separado a una quinta pregunta: **¿se sostiene igual dentro y fuera de muestra?** Cada una repite su propia prueba de cabecera sólo con las operaciones IS y sólo con las OOS, y las enseña una encima de la otra en el mismo histograma — no dos tablas que hay que comparar de memoria, sino la misma figura con dos distribuciones traslúcidas y las líneas de cada una (backtest, mediana, percentil de referencia) en su propio color. Es la forma de ver una degradación real sin necesidad de leer números sueltos.

Termina con un veredicto por estrategia — `STRONG`, `ACCEPTABLE`, `MARGINAL`, `FAIL` o `INCONCLUSIVE` — y, lo más importante, **con la lista de pruebas que falló y el número que la tumbó**.

Cuándo lo usas, y cuándo no

Úsalo cuando ya tienes un puñado de estrategias que han pasado los filtros anteriores y quieres saber cuáles son frágiles y por dónde. También sirve para una **cartera**: con `--portfolio` junta las operaciones de todas las estrategias en un solo flujo ordenado por tiempo y corre exactamente las mismas pruebas sobre él.

Cuándo NO sirve:

- **No detecta sobreajuste.** No hay Deflated Sharpe ni CSCV aquí, y no los va a haber: harían falta todas las estrategias que se probaron durante la generación, y en esta fase no existen. Fabricar ese número daría una cifra creíble y falsa. El sobreajuste se mira en el análisis de generación.
- **No valida el edge.** Da por hecho que lo tiene. Si lo corres sobre estrategias que no han pasado el decaimiento, un `MARGINAL` no significa casi nada.
- **No te dice qué riesgo usar**, aunque el techo de drawdown lo parezca: ese techo compara contra una cuenta de 100.000 $ arriesgando 1.000 $ por operación. Cambia el riesgo y cambia el veredicto.

Antes de empezar

- **Las operaciones tienen que estar exportadas** en `~/Desktop/AlgoData/raw/<proyecto>/<databank>/<fecha>/trades.parquet`, un solo archivo con todas las estrategias. Si no está: `python3 -m sqx.export.export_trades --project XAUUSD --databank Results --symbol XAUUSD_DukasM1_Infinox`.

- **Las barras del mercado** en la librería, de donde sale la volatilidad diaria que define los regímenes. No hace falta el M30: se calcula desde el minuto. Si el mercado no está en la librería: `python3 -m sqx.export.sync_bars` — ver el capítulo 13.
- **El activo tiene que tener ficha** en `assets/`. Corre antes `python3 -m core.assets XAUUSD` y léela. Aquí un `use: null` **no bloquea**, porque el coste de cada operación no se elige: se recupera del propio backtest (bruto menos neto), que es lo que SQX cobró de verdad.
- **No hace falta arrancar nada.** No toca SQX ni el worker: lee CSV. Puedes lanzarlo con la GUI abierta.

Cómo se ejecuta

```
python3 -m portfolio.common.monteCarlo.report --project XAUUSD --databank Results \
        --asset XAUUSD --day 2026-09-03
```

flag	obligatorio	qué hace
`--project`	sí	nombre del proyecto tal y como aparece en SQX
`--databank`	sí	nombre del databank, con sus espacios si los tiene
`--asset`	sí	nombre del activo en `assets/`, por ejemplo `XAUUSD`
`--day`	no	la fecha de la carpeta de exportación; hoy si no se pone, así que pásala si el export es de otro día
`--portfolio`	no	analiza todas las estrategias como una sola cartera
`--strategy`	no	una sola estrategia del export (`"Strategy 1.26.74"`): escribe solo `estrategias/<nombre>.json` y su página, **sin tocar** `verdict.csv` , `flags.csv` , `monteCarlo.json` ni el manifiesto. Es lo que hace «Run solo esta estrategia» en la ventana. Tarda lo de una estrategia (≈ 45 s con 100.000 simulaciones), no lo de todo el databank. No se combina con `--portfolio`
`--bars-timeframe`	no	de qué barras sale la volatilidad diaria; `M30` por defecto
`--set`	no	cambia cualquier valor del config: `--set global.n_sims=100000`

Todo lo ajustable está en `portfolio/common/monteCarlo/config.yaml`, agrupado por familia y con un comentario por línea. No hay ni un número escondido en el código. Los tres que importan:

ajuste	por defecto	qué cambia
`global.n_sims`	100.000	precisión de las colas y casi todo el tiempo de cálculo
`blocks.n_block_sizes`	6	cuántos tamaños de bloque se prueban en las familias A y B
`scoring.survival_dd_pct`	10%	el techo de drawdown que la cuenta aguanta. **Provisional**
`stability.n_stability_runs`	8	cuántas veces se recalculan los números que deciden

Cada estrategia corre entera en su propio proceso, las más largas primero, y los caminos se calculan en un kernel compilado. Medido el 2026-09-25 en esta máquina (96 núcleos): las 36 estrategias del ejemplo a **20.000** simulaciones, estabilidad incluida, en **27 segundos** (antes, 8 minutos). Con las 100.000 por defecto, cuenta unas cinco veces eso.

La comprobación de degradación IS/OOS de las familias A, B, C y D añade catorce remuestreos más por estrategia, a las mismas 100.000 simulaciones — en coste, como analizar una muestra dentro/fuera de la Familia B siete veces. Van al mismo grupo de procesos que todo lo demás, así que no se suman en serie: medido en una máquina de 16 núcleos, una estrategia completa (las cinco familias, con esto incluido) tardó 30 segundos — el tiempo lo decide cuántos núcleos tengas, no una cifra fija de esta página.

```
# XAUUSD — overrides to apply
SQX symbol: XAUUSD_DukasM1_Infinox   verified: 2026-09-03
- **spread**: use `UNDECIDED` points (SQX default `10.0`) —
- **commission**: use `UNDECIDED` $ per lot per side (SQX default `SizeBased 8`) —
- **point_value**: use `UNDECIDED` $ per 1.0 of price per 1.0 lot (SQX default `100.0`) —
36 streams · estabilidad sobre Strategy 18.27.27
[1/36] Strategy 1.17.44  (1470 operaciones)
  Barajado i.i.d.                              |######################| 100.0% 100,000/100,000    0s
  Bloques barajados — bloques de 5 operaciones |######################| 100.0% 100,000/100,000    0s
  Bloques barajados — bloques de 19 operaciones|######################| 100.0% 100,000/100,000    0s
  Bloques barajados — bloques de 32 operaciones|######################| 100.0% 100,000/100,000    0s
  Bloques barajados — bloques de 46 operaciones|######################| 100.0% 100,000/100,000    0s
  Bloques barajados — bloques de 59 operaciones|######################| 100.0% 100,000/100,000    0s
  Bloques barajados — bloques de 73 operaciones|######################| 100.0% 100,000/100,000    0s
  Bootstrap estacionario — bloques de 11 operaci|######################| 100.0% 100,000/100,000    0s
  Remuestreo i.i.d.                            |######################| 100.0% 100,000/100,000    0s
  Bloques remuestreados — bloques de 5 operacion|######################| 100.0% 100,000/100,000    0s
  Bloques remuestreados — bloques de 19 operacio|######################| 100.0% 100,000/100,000    0s
  Bloques remuestreados — bloques de 32 operacio|######################| 100.0% 100,000/100,000    0s
  Bloques remuestreados — bloques de 46 operacio|######################| 100.0% 100,000/100,000    0s
  Bloques remuestreados — bloques de 59 operacio|######################| 100.0% 100,000/100,000    0s
  Bloques remuestreados — bloques de 73 operacio|######################| 100.0% 100,000/100,000    0s
  Entradas perdidas                            |######################| 100.0% 100,000/100,000    0s
    ...
  Spread más ancho                             |######################| 100.0% 100,000/100,000    0s
    FAIL  compuesto 24  3 vetos
36 estrategias → /home/sergioguslw/Desktop/AlgoData/reports/XAUUSD/Results/2026-09-10/montecarlo
```

Qué produce

En `~/Desktop/AlgoData/reports/<proyecto>/<databank>/<fecha>/monteCarlo/` — y en `monteCarlo_portfolio/`, al lado y sin pisarlo, cuando corres con `--portfolio`. (Hasta el 25-09 las carpetas se llamaban `montecarlo/` y `montecarlo_portfolio/`; ver `docs/MAPA-DE-CARPETAS.md`.)

archivo	qué es
`monteCarlo.html`	**la página que hay que abrir.** Resumen del databank, qué falló y la tabla de todas
`estrategias/<nombre>.html`	el informe completo de una estrategia: las cinco familias con sus figuras
`estrategias/<nombre>.json`	ese mismo informe como resultado del contrato de estudios: lo que pinta la ventana
`verdict.csv`	una fila por estrategia: identidad, veredicto, compuesto, las cinco notas y cada número que decidió
`flags.csv`	una fila por prueba disparada: qué estrategia, qué prueba, con qué valor y si veta
`monteCarlo.md` / `monteCarlo.json`	el resumen escrito, y el mismo resumen como resultado del contrato
`manifest.json`	qué se analizó y con qué configuración entera

Los informes se acumulan por fecha: uno nuevo no borra el anterior.

Cómo se lee el resultado

Se lee **en este orden**, y el orden importa.

1. La página del databank: qué falló, antes que ninguna nota.

Monte Carlo — XAUUSD / Results

36 estrategias · 100,000 simulaciones por prueba · export 2026-09-03. Cada estrategia llega aquí ya con edge: esto mide de qué depende, no si existe.

36	2	40	0
estrategias analizadas	sin ningún veto	compuesto mediano	sin muestra suficiente para decidir

Lo que falló

Cada prueba que vetó o avisó, cuántas estrategias tumbó, y el número de una de ellas.

> **dead_block** — veto, 30 estrategias. Por ejemplo `Strategy 1.25.34` : Hay un bloque de 24 meses con mediana negativa (-6,080 $): un periodo entero en el que la estrategia no funcionó.

> **dd_99** — veto, 21 estrategias. Por ejemplo `Strategy 1.17.44` : El drawdown del percentil 99 es 12.8% de la cuenta, por encima del techo de supervivencia 10%. Con este riesgo por operación la cuenta no aguanta la peor de cien reordenaciones.

> **pf_5** — veto, 13 estrategias. Por ejemplo `Strategy 1.25.34` : El profit factor del percentil 5 es 1.00, por debajo de 1.10.

> **spread_widen** — veto, 4 estrategias. Por ejemplo `Strategy 1.25.34` : Con un spread más ancho el profit factor del percentil 5 baja a 1.07 (mínimo 1.10).

> **cost_shock** — veto, 3 estrategias. Por ejemplo `Strategy 1.25.34` : Con costes hasta el doble, el profit factor del percentil 5 baja a 1.09 (mínimo 1.10).

> **fill_degrade** — veto, 3 estrategias. Por ejemplo `Strategy 1.25.34` : Con ejecuciones peores el beneficio queda en el 75% del real (mínimo 75%).

> **high_vol** — veto, 1 estrategia. Por ejemplo `Strategy 1.25.34` : En el tercil de volatilidad alta la mediana remuestreada es -1,926 $: el sistema no gana en el régimen que más le va a tocar.

> **inflation_watch** — aviso, 36 estrategias. Por ejemplo `Strategy 1.17.44` : El drawdown reordenado es 2.2 veces el del backtest; por encima de 1.5 conviene vigilarlo, no descartarlo.

> **windows** — aviso, 36 estrategias. Por ejemplo `Strategy 1.17.44` : Sólo el 38% de las ventanas móviles aguanta en positivo en su percentil 5 (se pide 70%).

> **oos_red** — aviso, 30 estrategias. Por ejemplo `Strategy 1.25.34` : El Sharpe mediano fuera de muestra es el -36% del de dentro. Eso es optimismo que el test de decaimiento no recogió.

> **oos_amber** — aviso, 4 estrategias. Por ejemplo `Strategy 1.17.44` : El Sharpe fuera de muestra cae al 54% del de dentro (aviso por debajo de 60%).

Estrategia a estrategia

Pincha el nombre para ver su informe completo.

estrategia	veredicto	compuesto	A	B	C	D	E	operaciones	DD real	DD p95	inflaci
Strategy 17.18.29	MARGINAL	61	70	83	63	0	100	982	2.5%	4.2%	1
Strategy 18.28.28	FAIL	59	66	80	61	0	100	1,317	2.4%	4.4%	1
Strategy 17.17.41	MARGINAL	56	63	78	53	0	100	1,340	2.2%	4.4%	2
Strategy 16.20.43	FAIL	54	64	75	44	0	100	1,287	2.1%	4.3%	2

2. El informe de una estrategia: el veredicto, lo que falló, y sólo después los números.

Strategy 17.18.29

Robustez de una estrategia ya aceptada: cuánto de este resultado es suerte, y de qué tipo.

MARGINAL
veredicto

61
compuesto sobre 100

0
vetos disparados

982
operaciones

> Ninguna prueba veta. Lo que limita la nota es la familia D (0 sobre 100): ventanas que aguantan, el peor bloque de 24 meses, y el tercil de volatilidad alta. El compuesto es 61.

Lo que falló

> **aviso · familia A · inflation_watch** — El drawdown reordenado es 1.7 veces el del backtest; por encima de 1.5 conviene vigilarlo, no descartarlo.

> **aviso · familia B · oos_red** — El Sharpe mediano fuera de muestra es el 36% del de dentro. Eso es optimismo que el test de decaimiento no recogió.

> **aviso · familia D · windows** — Sólo el 48% de las ventanas móviles aguanta en positivo en su percentil 5 (se pide 70%).

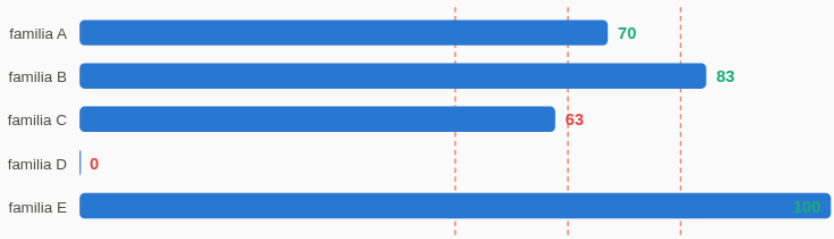
De un vistazo

El backtest contra la mediana de las simulaciones. La columna «rango» dice qué fracción de las simulaciones quedó por debajo del backtest: cerca del 100% es un backtest afortunado.

	backtest	mediana simulada	percentil 5	rango del backtest	confianza
beneficio neto	42,837 $	42,786 $	28,884 $	50%	reliable
Sharpe por operación	0.16	0.16	0.11	50%	reliable
drawdown máximo	2.46%	2.68%	1.72%	38%	reliable
Ret/DD	12.78	13.17	6.22	47%	reliable
racha perdedora	8.00	9.00	7.00	48%	reliable
inflación del drawdown	1.70×	—	—	—	reliable
PSR	1.0000	—	—	—	reliable

Puntuación por familia

0 a 100; las líneas son los cortes 80 / 65 / 50



- **A** — inflación del drawdown, margen del percentil 99 frente al techo de cuenta, y el Ret/DD del percentil 5
- **B** — beneficio y profit factor del percentil 5, y cuánto pesa la mejor operación
- **C** — la peor de las cuatro pruebas de ejecución

La columna «rango del backtest» dice qué fracción de las simulaciones quedó por debajo del backtest real. Cerca del 100% significa que el backtest fue de los buenos entre todos los mundos posibles: eso es suerte, no edge.

3. Familia A — el drawdown que no viste.

Familia A — suerte de orden

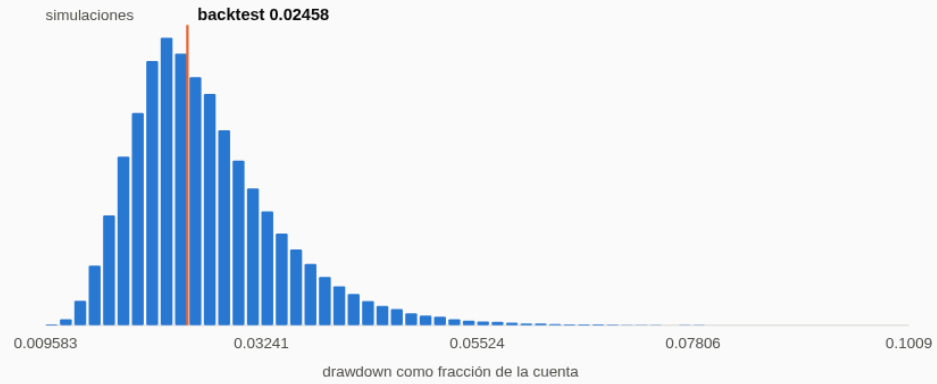
Las mismas operaciones, en otro orden. El beneficio no cambia; el drawdown y las rachas sí. Lo que mide es cuánto del drawdown cómodo del backtest fue el orden en que llegaron.

modelo (percentil 95)	beneficio neto	retorno	drawdown $	drawdown %	Re
backtest	42,837 $	42.84%	3,351 $	2.46%	1
Barajado i.i.d.	42,837 $	42.84%	4,960 $	4.18%	1
Bloques barajados — bloques de 5 operaciones	42,837 $	42.84%	4,915 $	4.16%	1
Bloques barajados — bloques de 14 operaciones	42,837 $	42.84%	4,321 $	3.69%	1
Bloques barajados — bloques de 23 operaciones	42,837 $	42.84%	3,591 $	3.09%	1
Bloques barajados — bloques de 31 operaciones	42,837 $	42.84%	4,421 $	3.75%	1
Bloques barajados — bloques de 40 operaciones	42,837 $	42.84%	3,650 $	3.17%	1
Bloques barajados — bloques de 49 operaciones	42,837 $	42.84%	3,728 $	3.20%	1
Bootstrap estacionario — bloques de 10 operaciones de media	56,255 $	56.25%	4,855 $	4.17%	2

> Comprobación: reordenar sin reemplazo movió el beneficio neto en 2.19e-11 $ — cero, como debe ser. Si no lo fuera, el modelo estaría cambiando la composición y no sólo el orden.

Drawdown máximo reordenando

percentil 95: 4.17% · backtest 2.46% · inflación 1.70×



A dónde llegaba la curva en otro orden

bootstrap estacionario — bloques de 10 operaciones de media, 200 puntos

bandas 5-95 y 25-75 · línea naranja: el backtest

156,982

La **inflación del drawdown** es el número clave: el percentil 95 del drawdown reordenado dividido por el que salió en el backtest. Por debajo de 1,5 es benigno; entre 1,5 y 3 hay que vigilarlo; por encima de 3 el drawdown del backtest fue una casualidad afortunada. **Dimensiona la cuenta con el percentil 95, no con el drawdown del backtest.**

La comprobación de la caja gris tiene que decir que barajar movió el beneficio en ~0 $. Si dice otra cosa, el modelo está cambiando la composición y no sólo el orden: no sigas leyendo.

Al final de cada familia (A, B, C y D) hay un apartado **«Degradación dentro / fuera de muestra»** con el histograma solapado descrito arriba. Léelo así: si la mediana y el percentil de referencia de la muestra OOS quedan muy por debajo de los de la IS, esa familia se está sosteniendo peor fuera de muestra que dentro — información que el veredicto agregado no distingue. *(Captura pendiente: el histograma solapado es nuevo y todavía no tiene imagen en* `assets/`*.)*

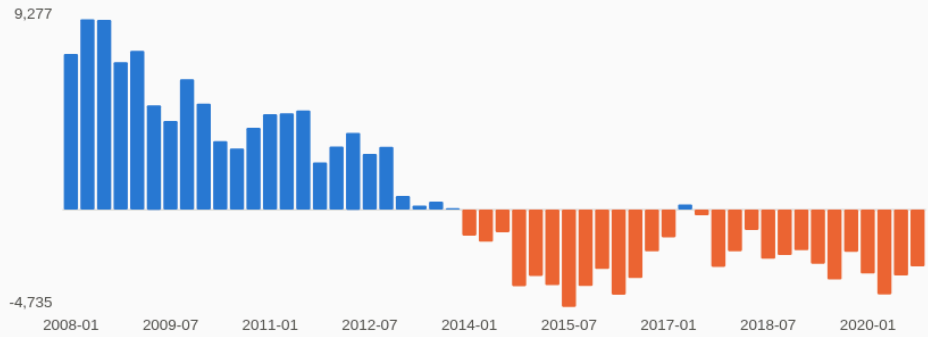
4. Familia D — dónde vivía el edge.

Familia D — suerte de régimen

Las familias A a C dan por hecho que el edge es el mismo siempre. Ésta lo ataca: mira dónde vivió, en el calendario y en la volatilidad del mercado.

Ventanas móviles de 24 meses

percentil 5 del beneficio en cada ventana · 48% en positivo



Bloques que no se solapan

Las ventanas solapadas suavizan; estos bloques no comparten ni una operación, así que un periodo malo de verdad aparece aquí y no allí.

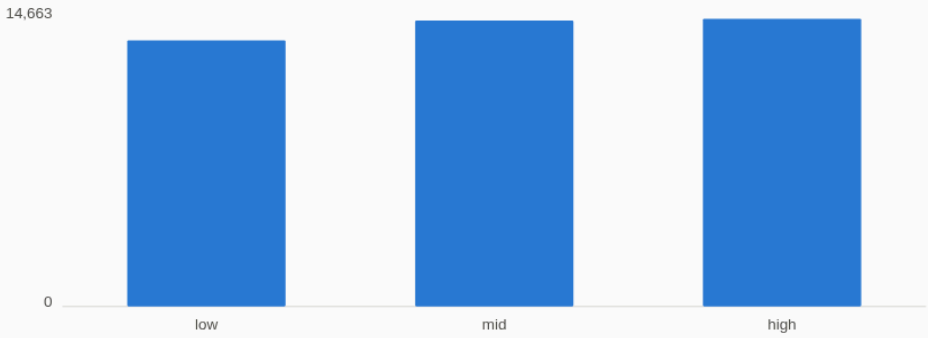
desde	operaciones	mediana	percentil 5	PF p5
2008-01-01	161	13,511 $	8,040 $	1.70
2010-01-01	163	9,672 $	5,252 $	1.49
2012-01-01	139	8,693 $	3,050 $	1.25
2014-01-01	133	3,251 $	-1,187 $	0.90
2016-01-01	94	1,813 $	-2,425 $	0.75
2018-01-01	105	2,680 $	-1,875 $	0.83
2020-01-01	111	1,038 $	-3,134 $	0.73

Régimen de volatilidad

ATR(20) diario, en terciles. Cortes en 18.13 y 22.72; cobertura 100.0%.

Beneficio mediano remuestreado por tercil de volatilidad

el 34% del beneficio sale de un tercil



tercil	operaciones	beneficio real	mediana	percentil 5	PF p5
low	327	13,747 $	13,562 $	5,932 $	1.22
mid	327	14,325 $	14,573 $	6,764 $	1.25
high	328	14,765 $	14,663 $	6,337 $	1.21

El peor camino posible

> Cosiendo un mal tramo de cada bloque de 24 meses — el percentil 5 de cada uno — el drawdown llega a **10.0%** de la cuenta, frente al 2.5% real y al 4.2% del percentil 95 reordenando. No es un veto: es la referencia de estrés.

Las barras naranjas son ventanas de dos años en las que la estrategia, remuestreada, pierde. Un bloque de 24 meses que no se solapa con ningún otro y sale en negativo es un veto: es un periodo entero en el que el sistema no funcionó, y las ventanas solapadas lo camuflan.

Debajo de esa tabla está la **curva de equity real con cada bloque marcado** — la misma curva del backtest, con una línea discontinua donde empieza cada bloque de la tabla de arriba, para ver a ojo en qué tramo del calendario vive el bloque muerto. Y en el régimen de volatilidad, el precio y la serie de volatilidad (ATR o GARCH, la que esté activa) superpuestos sobre el mismo eje de tiempo, con el fondo coloreado por tercil — pasa el ratón por la línea de precio o la de volatilidad para resaltar cada una por separado. *(Captura pendiente para ambas figuras.)*

Qué valor es bueno, resumido:

número	bien	vigilar	mal
inflación del drawdown	≤ 1,5	1,5 – 3	> 3
beneficio del percentil 5	> 0 y con margen	cerca de 0	≤ 0
profit factor del percentil 5	> 1,20	1,10 – 1,20	≤ 1,10
Sharpe OOS / Sharpe IS	> 0,60	0,40 – 0,60	< 0,40
ventanas móviles en positivo	> 70%	50 – 70%	< 50%
PSR	≥ 0,95	0,90 – 0,95	< 0,90

La PSR se lee sólo sobre las operaciones fuera de muestra (dueño, 2026-09-29): el build ya seleccionó la estrategia, así que comparar contra el operador aleatorio ahí mediría la búsqueda, no el edge. Con menos operaciones OOS de las que hacen falta para fiarse de una media, la PSR sale en blanco y aparece un veto `oos_thin`: el veredicto se queda en INCONCLUSIVE, nunca se rellena con las operaciones del build.

Un ejemplo completo

Las 36 estrategias del databank `Results` del proyecto `XAUUSD`, con la configuración por defecto:

```
python3 -m portfolio.common.monteCarlo.report --project XAUUSD --databank Results \
        --asset XAUUSD --day 2026-09-03
```

El resumen escrito que deja en `monteCarlo.md`:

```
# Monte Carlo — XAUUSD / Results

36 estrategias · 100,000 simulaciones por prueba · export 2026-09-03

## Veredicto

| veredicto | estrategias |
|---|---|
| FAIL | 34 |
| MARGINAL | 2 |

Pasan sin ningún veto: **2** de 36.

## Por qué caen las que caen

- `dead_block` — 30 estrategias
- `dd_99` — 21 estrategias
- `pf_5` — 13 estrategias
- `spread_widen` — 4 estrategias
- `cost_shock` — 3 estrategias
- `fill_degrade` — 3 estrategias
- `high_vol` — 1 estrategia

## Estabilidad del propio Monte Carlo

Con 100,000 simulaciones y 8 repeticiones independientes, el número que más se mueve es
`iid_bootstrap.dd_pct.99`, con una dispersión del 1.3% de su media. Por debajo de la
tolerancia, así que las cifras que deciden son estables.
```

Lo que hay que leer de ahí, en cristiano:

- **Ninguna es sólida.** Dos se quedan en `MARGINAL` y el resto tienen algún veto.
- **`dead_block` tumba a casi todas.** 30 de 36 tienen un bloque de dos años en el que pierden. Mirando las ventanas móviles se ve por qué: **el edge se muere a partir de 2013**, y el resultado total lo sostienen los primeros años. Eso no lo enseña ninguna métrica del databank.
- **`dd_99` tumba a 21 de 36** con el techo del 10%. Ese techo es provisional y es tanto una afirmación sobre el **riesgo por operación** como sobre las estrategias: con 500 $ por operación en vez de 1.000 $, la mitad lo pasan. Mientras no estén las reglas de la prop firm, ese veto se lee como «con este riesgo, no».
- **El optimismo dentro de muestra es enorme.** El Sharpe mediano fuera de muestra es el **14%** del de dentro (mediana de las 36). Eso no es sobreajuste medido — es una comparación de nivel — pero es optimismo que el test de decaimiento no había recogido.
- **Cuenta con el doble de drawdown del que viste.** La inflación mediana es **1,95**, y la más baja de las 36 es 1,52. Ninguna tiene un drawdown de backtest que sirva para dimensionar la cuenta.

Una estrategia cada vez: la ventana

El panel del navegador (`explorer.serve`, puerto 8765) **se retiró el 25-09**. Lo que hacía — una estrategia cada vez, con los knobs a mano — lo hace ahora la ventana de escritorio (`ui/`): llama a `portfolio.common.monteCarlo.one` para la estrategia elegida, con los `--set` del cajón de configuración, y pinta el diccionario que devuelve. El comando y la ventana usan el mismo `one.run`, así que la ficha de `estrategias/<nombre>.html` y lo que ves en la ventana no pueden discrepar. Las frases del cajón salen de `portfolio/common/monteCarlo/tooltips.py`.

Lo que el panel tenía sigue en el resultado: la pestaña **Explorador de pruebas** lleva cada sub-prueba × cada estadístico ya calculado, con dos selectores (`prueba`, `métrica`) que la ventana filtra sin volver a calcular, más el cono de equity de cada sub-prueba. **Re-ejecutar una sub-prueba** es `one.run(..., only="<etiqueta>")`: devuelve esa sub-prueba sola, con azar nuevo y sin veredicto — un veredicto sale de un análisis entero o no sale —, y la ventana la pone al lado de la guardada.

Qué NO te dice

- **Que una estrategia con `MARGINAL` funcione.** Dice que ninguna de estas pruebas la tumbó. La prueba de que funciona es el mercado.
- **Nada sobre sobreajuste.** Ver arriba. Es el error más fácil de cometer con un Monte Carlo.
- **Nada que se repita exacto dos veces.** No hay semilla, a propósito: cada tirada usa entropía nueva. Lo que sí te da es la **dispersión** entre tiradas independientes (sección de estabilidad); si es grande, la respuesta es subir `n_sims`, no fijar una semilla y creerte la primera.
- **Nada sobre la cartera**, salvo que la pidas con `--portfolio`. Doce estrategias que pasan por separado pueden hundirse a la vez.

Si algo falla

- `FileNotFoundError` en `bars/<feed>/M1.parquet` — ese mercado no está en la librería de barras. `python3 -m sqx.export.sync_bars`.
- `ValueError: N trades open on days the bar file does not have` — las barras no son del mismo feed que las operaciones, o les falta histórico. No se imputa nada a propósito: compararías la estrategia con los regímenes de otro instrumento.
- `FileNotFoundError ... trades.parquet` al empezar — esa fecha de exportación no existe, o es de antes de que los trades se empaquetaran. Comprueba la fecha: es la de la exportación, no la de hoy.
- Un aviso de que el coste modelado no cuadra con el recuperado — la ficha de `assets/` describe otro instrumento. Sólo invalida la familia C; el resto del informe sigue en pie.

32. Configurar el MC Retest en SQX — las ocho tareas, escritas de una vez

> **Ojo con el nombre.** Esta página es la mitad de SQX del paso 13: **dejar las ocho tareas configuradas** dentro de un custom project. Leer los resultados es la página `08-montecarlo.pdf › 11-retest-mc` (paso 14), y el Monte Carlo que reordena operaciones es la `08-montecarlo.pdf › 07-montecarlo`, que es otra cosa.

Qué pregunta responde

El MC Retest vuelve a correr el backtest entero mil veces cambiando **una sola cosa** cada vez: el spread, el slippage, los parámetros, las velas. Ocho tareas, ocho preguntas distintas, y por eso se sabe **cuál** de ellas rompe la estrategia.

Configurarlas a mano son ocho formularios de SQX con seis pestañas cada uno, y basta que en uno se quede un método de más encendido para que esa tarea deje de poder atribuir nada. Este comando las escribe las ocho desde el catálogo de `assets/_build.yaml`, con los rangos de perturbación del activo, y dice en voz alta lo que ha dejado sin configurar y por qué.

Cuándo lo usas, y cuándo no

Úsalo cuando ya tienes un custom project con su población dentro — después de la puerta OOS, del crossmarket y del crossTF — y quieres dejarlo listo para lanzar el MC Retest.

No lo uses sobre el `Builder` ni el `Retester` de serie (regla dura 10), ni sobre una población recién construida sin cribar: mil simulaciones por estrategia sobre diez mil estrategias es una factura de CPU que no se paga para descartar lo que la puerta OOS descarta gratis.

La regla de la distancia mínima

`MCR 4 MinDist` sólo se configura si la población lleva órdenes stop o limit. Decisión del dueño, 2026-09-23, y es simple aritmética: la distancia mínima es lo cerca del precio que un bróker te deja poner una orden **pendiente**. Si todas las entradas son a mercado no hay ninguna orden esperando en el libro, así que esa tarea sortearía un número que nadie lee, y mil simulaciones después diría que la estrategia es perfectamente robusta a algo que no le pasa.

El comando lo mira solo, en este orden:

1. **La población**, si el databank ya está construido: abre los `.sqx` y lee qué órdenes usan de verdad. Se para en cuanto encuentra la primera pendiente. 0,2 ms por estrategia.
2. **El generador**, si todavía no hay población: si la tarea Build no tiene encendidos `EnterAtStop` ni `EnterAtLimit`, la respuesta es no y es segura — lo que el generador no puede emitir no puede estar en el databank.
3. Si el generador **sí** puede emitirlas y aún no hay estrategias, la pregunta **no está resuelta**, y el comando no se la inventa: deja la tarea sin configurar y te dice que vuelvas a pasarlo después del build.

Lo mismo vale dentro de `MCR 8 Stress`, que lleva seis métodos a la vez: si no hay órdenes pendientes se cae **ese método** y la tarea se escribe con los otros cinco. La condición es del método, no de la tarea.

Las tres cosas que se tocan por tarea

Todo está en `assets/_build.yaml`, bloque `mc_retest:`, y nada de esto vive en el código.

`simulations` — cuántas veces se vuelve a correr el backtest perturbado. **Cada tarea lleva el suyo**, así que se puede abaratar una cara sin tocar las demás: `MCR 7 OHLC` a 250 y el resto a 1.000 es una línea. El coste es lineal — la mitad de simulaciones, la mitad de tiempo — y por debajo de unas 200 el percentil de confianza que lee el paso 14 empieza a ser ruido.

`precision` — es `MCBacktestPrecision`, la fidelidad intrabarra de cada simulación, y **no** es el `testPrecision` del backtest principal. Los dos valores van al revés de lo que sugiere la palabra:

valor	qué es	coste medido
`1`	**sólo el timeframe elegido.** Rápido y grosero: ve las velas M30 tal cual, sin mirar qué pasó dentro	20–25 s
`2`	**un minuto.** Lento y fino: reconstruye cada vela desde barras de un minuto, así que respeta el orden real en que se tocaron máximo y mínimo	50 s

Mismas 1.000 simulaciones y misma ventana en los dos casos, medido el 2026-09-23. O sea: el número alto es el preciso, y cuesta el doble.

`segment` — qué ventana se vuelve a correr. Dos formas y sólo dos:

```
segment: build          # un tramo suelto, tal como lo define assets/_policy.yaml
segment: build..oos1    # DOS PUNTOS: una sola ventana continua que abarca los dos
```

`build..oos1` **no son tres puntos** y no es "los dos tramos por separado": es una ventana continua del principio del primero al final del último —en XAUUSD, 2008.01.01 → 2022.12.31— y además marca el segundo tramo como fuera de muestra dentro de la tarea, que es lo que hace que `full_sample: true` signifique algo. Se cobra a los costes del **último** tramo, o sea al spread de `oos`, el más caro de los dos declarados: una ventana que abarca las dos muestras se lee al precio pesimista.

`oos2` **vale** también, suelto o dentro de un rango (dueño, 2026-09-28). Sólo a un agente autónomo (`ALGO_AUTONOMOUS=1`) se le niega el comando.

Antes de empezar

- El proyecto tiene que existir y ser **custom** (`sqx.projects.builder`), clonado del donante congelado, que es el que trae las ocho tareas con sus nombres.
- El `.cfx` **no puede estar abierto** por una instancia de SQX: SQX lo reescribe al salir y se lleva el cambio por delante sin decir nada (regla dura 4). El comando lo comprueba y se niega.
- El activo tiene que tener sus costes pactados (`python3 -m core.assets <SIMBOLO>`), y sus rangos de perturbación decididos en `assets/symbols/<SIMBOLO>.yaml`, bajo `mc_retest:`. Un rango en `null` no bloquea nada: sólo se cae el método que lo necesitaba, y el comando lo canta.

No toca SQX ni ningún worker: sólo lee y reescribe un fichero. Tarda menos de un segundo.

Cómo se ejecuta

```
python3 -m sqx.projects.mcretest XAUUSD \
    --cfx ~/Desktop/SQX_w2/user/projects/XAU_estudio/project.cfx \
    --input OOS
```

flag	obligatorio	qué hace
`XAUUSD`	sí	el activo, para sus costes, sus ventanas y sus rangos de perturbación
`--cfx`	sí	el `project.cfx` del custom project
`--input`	sí	el databank que leen **las ocho**. No se adivina: es la población que se perturba
`--databank-dir`	no	la carpeta de ese databank, para mirar qué órdenes lleva. Por defecto, la del propio proyecto
`--json`	no	saca el resultado entero como JSON, para encadenarlo

Qué produce

Reescribe **el mismo** `project.cfx`, en sitio. Por cada tarea del catálogo:

- enciende el crosscheck `MonteCarloRetest` y deja activo **sólo** el método que le toca;
- escribe las mil simulaciones, la precisión y si corre sobre muestra completa;
- pone la ventana y los costes del tramo que esa tarea usa;
- **apaga todas sus condiciones de aceptación**;
- apunta su databank de entrada al que le has dicho.

La tarea que no se puede configurar se queda **inactiva y con el crosscheck apagado**, para que no pueda correr por descuido pareciendo lo que no es.

Cómo se lee el resultado

Salida real, sobre una copia del donante congelado y con la población del `OOS` del maestro:

```
XAUUSD  XAUUSD  <- OOS
  ordenes (population, 231 estrategias leidas): EnterAtMarket  ->  pendientes: no
  ✓ MCR 1 Bar      RandomizeStartingBar  build 2008.01.01→2017.12.31  1000 sims  1
condicion(es) apagada(s)
  ✓ MCR 2 Spread   RandomizeSpread  build 2008.01.01→2017.12.31  1000 sims  1 condicion(es)
apagada(s)
  ✓ MCR 3 Slippage RandomizeSlippage  build 2008.01.01→2017.12.31  1000 sims  1 condicion(es)
apagada(s)
  ⊘ MCR 4 MinDist  NO configurada — la poblacion no lleva ordenes stop ni limit
  ✓ MCR 5 Params   RandomizeStrategyParameters  build 2008.01.01→2017.12.31  1000 sims  1
condicion(es) apagada(s)
  ✓ MCR 6 Exits    RandomizeExitParameters  build 2008.01.01→2017.12.31  1000 sims  1
condicion(es) apagada(s)
  ✓ MCR 7 OHLC     RandomizeHistoryDataOHLC  build 2008.01.01→2017.12.31  1000 sims  1
condicion(es) apagada(s)
  ✓ MCR 8 Stress
RandomizeExitParameters+RandomizeHistoryDataOHLC+RandomizeSlippage+RandomizeSpread+RandomizeStr
ategyParameters  oos1 2008.01.01→2022.12.31  1000 sims  1 condicion(es) apagada(s)
      ⚠ sin RandomizeMinDistance: la poblacion no lleva ordenes stop ni limit

Las ocho leen el MISMO databank: son ocho lecturas de una poblacion, no un embudo.
El veredicto se toma en Python (studies/breakage/mcRetest/) y se aplica con /curate.
```

Línea por línea:

`ordenes (population, 231 estrategias leidas)` — de dónde salió la respuesta. `population` significa que abrió los `.sqx` de verdad; `generator` significa que todavía no hay población y se ha mirado qué puede emitir el builder.

`⊘ MCR 4 MinDist` — la regla del dueño, aplicada. Esta población entra a mercado, así que esa tarea no existe para este estudio.

`build 2008.01.01→2017.12.31` — las siete aisladas vuelven a correr **la muestra de construcción**: preguntan si el ajuste del build fue frágil. La de estrés va sobre `oos1`, que aquí significa la muestra entera 2008–2022 con el tramo 2018–2022 marcado como fuera de muestra, y se cobra al spread de `oos`, que es el más caro de los dos declarados.

`1 condicion(es) apagada(s)` — la que convertía la tarea en un filtro. Con ella encendida SQX borra lo que falla, cada tarea encadena en la siguiente y al final sólo sabes cuántas sobrevivieron: nunca **a qué** murió cada una, que es lo único que ocho tareas aisladas existen para responder.

Lo que cuesta correrlo

Medido el 2026-09-23 en el custodio (48 núcleos), **2 estrategias × 1.000 simulaciones por tarea**:

tarea	precisión MC	tiempo
MCR 1 Bar	1	25 s
MCR 2 Spread	2	50 s
MCR 3 Slippage	2	50 s
MCR 5 Params	1	20 s
MCR 6 Exits	2	50 s
MCR 7 OHLC	2	**149 s**
MCR 8 Stress	2	**227 s**
total		**9 min 31 s**

Dos cosas que decidir antes de lanzarlo sobre una población de verdad:

- **Escala con el número de estrategias.** Diez minutos son dos estrategias; cien son unas ocho horas. Esto va al final del embudo, sobre supervivientes, nunca sobre una población recién construida.
- **La precisión del MC es el factor dos.** Las dos tareas a precisión 1 tardan la mitad que sus hermanas a precisión 2 con el mismo trabajo. `MCR 7 OHLC` es cara por otro motivo: reconstruye el histórico entero en cada simulación. Y la de estrés, además, corre 15 años en vez de 10.

Un ejemplo completo

```
$ python3 -m core.assets XAUUSD          # preflight obligatorio (regla dura 5)
...
AVISO: los rangos MC Retest min_distance no están decididos. No bloquea,
pero esa tarea del MC Retest no es interpretable hasta que el dueño los fije.

$ python3 -m sqx.projects.mcretest XAUUSD --cfx
~/Desktop/SQX_w2/user/projects/XAU_estudio/project.cfx --input OOS
```

Y después, para lanzarlo:

```
bin/sqx-worker.sh --role custodian start
python3 -c "from core import worker; worker.call('-project action=start
name=XAU_estudio','custodian')"
```

Una sola cosa a la vez en el custodio, y `bin/sqx-worker.sh --role custodian stop` al acabar.

Desde el 2026-10-07: qué corre el paso 13 (T1 y tres tareas de SQX)

> **Estado:** rediseño decidido por el dueño el 2026-10-07. **Aún no se ha probado en vivo en SQX**: ni el lote de vecinas, ni sus tiempos, ni el configurador sobre un proyecto sin las cinco tareas retiradas. No hay captura de salida real todavía: la primera ejecución real está pendiente.

El paso 13 ya no corre las ocho tareas. Corre, por este orden en el autopiloto (juez del 12 → vecinas → SQX 13 → Python 14 → juez del 14 → SQX 15):

1. **T1, las vecinas de parámetros.** No es una tarea de MC Retest (SQX guarda por simulación sólo el vector de ganancias en céntimos, sin fechas: `knowhow/sqx-format/mc-retest-storage.md`), sino un lote por la fábrica de variantes. Cada madre se vuelve a probar con hasta 500 vecinas con **todos** sus parámetros declarados movidos a la vez (salvo desplazamientos y selectores), y se reprueba en el tramo de build y en oos1 (nunca oos2). Sus operaciones se exportan y se valoran como en el paso 8: reponderador común menos la deriva por año natural. De ese lote salen dos hechos sin umbral (2026-10-12): el percentil de la madre entre sus vecinas en oos1 (por exceso neto de deriva) y el p99 del drawdown máximo de las vecinas sobre build + oos1 juntos, en R (columna `dd_r` de `neighbours.parquet`; 1R es el riesgo fijo por operación de la doctrina). Salen en la pestaña «Fragilidad» del paso 14.
2. **Tres tareas de SQX:** `MCR 1 Bar`, `MCR 4 MinDist` y `MCR 7 OHLC`. Quedan **fuera de la etapa** `MCR 2 Spread`, `3 Slippage`, `5 Params`, `6 Exits` y `8 Stress` (`sqx/projects/stages.yaml`); su definición sigue en `assets/_build.yaml`.

```
# 1) fabricar el lote: cada madre (vecina 0, intacta) y sus vecinas, en una carpeta
python3 -m sqx.variants.neighbourbatch --mothers <carpeta de .sqx> --out <lote>
# 2) reprobarlo en las dos patas (con el custodio arrancado, regla 18: una sola espera)
python3 -m sqx.variants.neighbourbatch --work <lote> --project <Proyecto>
# 3) valorar las operaciones y guardar la tabla por vecina
python3 -m studies.breakage.mcRetest.neighbours.t1 --project <Proyecto> --work <lote> \
    --symbol XAUUSD --timeframe H1
```

flag	de qué comando	qué hace
`--mothers`	neighbourbatch	carpeta con los `.sqx` de las madres: fabrica
`--out`	neighbourbatch	carpeta del lote a fabricar
`--work`	neighbourbatch / t1	lote ya fabricado: reprobarlo / leer sus operaciones
`--project`	ambos	el custom project que tiene las patas WFC
`--symbol`, `--timeframe`	t1	activo y marco, para valorar con sus costes y barras
`--day`	t1	fecha de la tabla; por defecto, hoy

Todo se ejecuta con el worker parado salvo el paso 2, que lo lanza el autopiloto. Parámetros y dónde viven:

parámetro	valor inicial	dónde
`n` vecinas por madre (tope, no cuota: los duplicados se descartan)	500	`sqx/variants/config.yaml › neighbours`
`entry_pct` movimiento de entradas e indicadores	±15 %	idem
`exit_pct` movimiento de salidas	±30 %	idem
`fixed` nombres que no se mueven (desplazamientos, selectores)	dos expresiones	idem
`seed` , `int_floor`	0, 1	idem
`legs` patas reprobadas	WFC 1 IS, WFC 2 OOS1	idem (nunca WFC 3 OOS2)
`shape`	minimal	idem
niveles de la banda, `cvar_alpha` , remuestreos, confianza	5-95, 0,05, 9999, 0,95	`studies/breakage/mcRetest/config.yaml › fragility`
deriva (`drift.window` , `drift.variant`)	los del paso 8	`studies/screening/gate/config.yaml` (T1 no tiene los suyos)

Proyectos viejos (construidos con la cadena antigua): su `SPP IS` sigue leyendo `MCR 8 Stress` , que ya nada rellena. Antes del paso 15, con el worker parado: `python3 -m sqx.projects.spp XAUUSD --cfx <cfx> --input "MCR 7 OHLC"` .

Qué NO te dice

- **No dice si la estrategia es buena.** Deja las tareas configuradas; quien las corre es SQX y quien las lee es `studies/breakage/mcRetest/` .
- **No decide nada sobre la población.** Las condiciones de aceptación se apagan a propósito: esto produce evidencia, y el veredicto se toma después, en Python, y se aplica con `/curate` .
- **No comprueba que los rangos tengan sentido.** Que el spread se sortee entre 5 y 30 puntos es una decisión escrita en `assets/symbols/XAUUSD.yaml` ; el comando la escribe, no la juzga.
- **No sustituye al build.** Si el proyecto no tiene población, el comando se configura igual, pero la pregunta de las órdenes pendientes puede quedarse sin resolver y lo dice.

Si algo falla

`el custodian tiene este proyecto abierto y reescribe el .cfx al salir` — para el worker antes: `bin/sqx-worker.sh --role custodian stop` . Es la regla dura 4 y no es negociable.

`el proyecto no lleva la tarea MCR 3 Slippage` — el proyecto no se clonó del donante congelado, o se creó con `--only` y se quedó sin esas tareas. Se vuelve a crear con `sqx.projects.builder` quedándose las `Retest` .

`rango min_distance sin decidir en assets/symbols/XAUUSD.yaml` — hay órdenes pendientes en la población, así que la tarea sí hace falta, pero nadie ha dicho entre qué dos distancias sortear. Ese número lo decides tú y va en el fichero del activo.

`sin poblacion construida: el generador PUEDE emitir ordenes pendientes` — el builder tiene `EnterAtStop` o `EnterAtLimit` encendidos pero todavía no hay estrategias. Corre el build y vuelve a pasar el comando; entonces la respuesta sale de la población y no de una suposición.

11. Retest de Monte Carlo — ¿habrían ocurrido siquiera estas operaciones?

> **Ojo con el nombre.** Aquí «retest» significa **Monte Carlo Retest**: SQX vuelve a ejecutar el backtest entero mil veces contra una entrada perturbada. No es el *retest multi-mercado*, que es la página `07-otros-mercados-y-timeframes.pdf › 05-retest-mercados`. Son dos cosas distintas con el mismo nombre.

Qué pregunta responde

Todas las pruebas de robustez que ya usas remezclan una lista de operaciones que **ya existe**: el Monte Carlo de operaciones las reordena, el retest multi-mercado las coloca en otro mercado. La estrategia nunca se vuelve a ejecutar.

Esta es al revés. Cada simulación cambia una entrada del backtest —el spread, un parámetro, la vela histórica— y **vuelve a correr la estrategia entera** contra esa entrada. Por eso tarda, y por eso responde algo más profundo: no «¿fue suerte la forma de esta curva?», sino «¿habría existido esta curva?». Y como cada tarea cambia **una sola cosa**, te dice **cuál** de ellas la rompe.

Cuándo lo usas, y cuándo no

Úsalo cuando una estrategia ya ha pasado los filtros de población, el decaimiento IS/OOS y el retest multi-mercado, y lo que quieres saber es de qué depende: del coste de entrar, del relleno que te dan, de sus propios parámetros, o del histórico exacto que le tocó ver.

No lo uses para decidir si la ventaja existe — eso lo dan otros estudios y aquí se da por supuesto. Y no lo uses para medir sobreajuste de generación: para eso hace falta saber cuántas estrategias se probaron, un número que en esta fase no existe.

Antes de empezar

Este comando **solo lee ficheros de disco**. No toca SQX, ni el worker, ni ningún proyecto, así que puedes lanzarlo con la GUI abierta sin ningún riesgo.

Lo que sí tiene que existir antes son **las ocho tareas ya corridas en SQX**, cada una en su propio databank del proyecto, con un único método de perturbación activo:

databank	qué aleatoriza	papel
`MCR 1 Bar`	la barra en la que empieza el backtest	**control**
`MCR 2 Spread`	el spread cobrado	ejecución
`MCR 3 Slippage`	el slippage del relleno	ejecución
`MCR 4 MinDist`	la distancia mínima de una orden pendiente al precio	ejecución
`MCR 5 Params`	todos los parámetros de la estrategia	especificación
`MCR 6 Exits`	solo los parámetros de salida	especificación
`MCR 7 OHLC`	el propio histórico de precios	datos
`MCR 8 Stress`	las seis cosas a la vez, sobre muestra completa	producción

Si un databank lleva dos métodos activos, el comando **se niega a escribir**. Es a propósito: una tarea que perturba dos cosas no sirve para atribuir el daño a ninguna.

Cómo se ejecuta

```
python3 -m studies.breakage.mcRetest.ingest --project XAUUSD --databank MCR_All --day 2026-09-18
```

flag	obligatorio	qué hace
`--project`	sí	nombre del proyecto tal y como aparece en SQX
`--databank`	no	nombre de **esta corrida de ingesta**, no de un databank de SQX. Por defecto `MCR_All`. Los ocho databanks de SQX van dentro como particiones
`--day`	no	fecha del export, `YYYY-MM-DD`. Por defecto hoy
`--limit`	no	solo las primeras N estrategias de cada tarea, para una prueba rápida
`--set`	no	cambia cualquier ajuste del `config.yaml`, p. ej. `--set ingest.capital=50000`

Tarda 2 segundos con 5 estrategias × 8 tareas × 1.000 simulaciones en esta máquina (medido el 2026-09-25, 1,9 GB de memoria real). Cada proceso escribe él mismo el P&L de su estrategia, así que nada grande viaja al proceso principal. `ingest.workers` (16 por defecto) es el mando que cambia memoria por tiempo: 40 procesos lo bajan a 1,4 s pero suben a 3,4 GB.

Qué produce

Todo bajo `~/Desktop/AlgoData/raw/<proyecto>/<databank>/<fecha>/`:

carpeta	qué es
`sims/`	**la tabla principal**: una fila por simulación con las 30 métricas reconstruidas
`levels/`	la tabla de confianza que guardó SQX, en formato largo
`original/`	el backtest sin perturbar del que salió cada tarea
`pnl/`	el P/L crudo de cada operación de cada simulación
`manifest.json`	qué métodos corrió cada tarea, con qué números, y el resultado de la reconciliación

Un export es inmutable. Si vuelves a lanzarlo sobre la misma fecha, el comando se niega en vez de escribir encima. Para rehacerlo, otra `--day` o borra la carpeta a mano.

Cómo se lee el resultado

La salida real de la corrida del 18 de septiembre de 2026:

```
ingest: 40 runs, 39996 simulations ->
/home/sergioguslw/Desktop/AlgoData/raw/XAUUSD/MCR_All/2026-09-18
  reconciled 30 metrics against SQX, 0 disagreements
  level tables unusable (run cut short): 3 — bar/1.19.29, exits/1.19.29, ohlc/41.5.25
```

Tres líneas, y las tres importan:

`40 runs, 39996 simulations` — 8 tareas × 5 estrategias. Si el número de simulaciones no es múltiplo redondo de 1.000, alguna corrida se cortó (ver la tercera línea).

`reconciled 30 metrics against SQX, 0 disagreements` — la línea que hace fiable todo lo demás. El estudio entero es una reconstrucción: SQX guardó once percentiles por métrica y tiró las simulaciones individuales, así que cada número de aquí está recalculado desde el P/L crudo. Esa reconstrucción solo vale si reproduce lo que SQX calculó. Son **12.210 comprobaciones** —30 métricas × 11 niveles × 5 estrategias × 8 tareas— y **si una sola falla, el comando no escribe nada.**

`level tables unusable` — estas tres corridas se cortaron antes de las 1.000 simulaciones. Las simulaciones que guardaron están bien (los índices se truncan por la cola, nunca dejan huecos), pero SQX escribió su tabla de confianza sobre las 1.000 que le pediste, así que **en esas tres la tabla guardada tiene todos los rangos desplazados** y no se lee. La reconstruida sí vale.

Métricas para criterios — sin umbral todavía

Desde el 2026-10-04 la pestaña «Las ocho tareas» lleva una tabla más, y la tabla de Databanks tres columnas del estrés combinado. Responden a «¿el backtest original es un resultado normal de lo que la tarea simuló, o un caso extremo?». **Ninguna tiene umbral ni cambia el veredicto.**

- **Percentil del original** (`percentil_original_estres`): qué porcentaje de las re-ejecuciones ganó menos que el backtest original. 50 = el original está en el centro; cerca de 100 = casi ninguna re-ejecución llegó a su resultado.
- **Brecha de la mediana (%)** (`brecha_mediana_pct_estres`): (mediana de las re-ejecuciones − original) / |original|. −30 = la re-ejecución típica gana un 30 % menos que el original. Tu ejemplo — real 14.000, mediana 3.500–4.000 — es una brecha de −73 %. Queda vacía si el original está demasiado cerca de cero para dividir por él.
- **Distancia a la mediana (IQR)** (`distancia_IQR_estres`): la misma distancia medida en rangos intercuartílicos de las re-ejecuciones (1 IQR ≈ 1,35 sigmas de una normal), que no depende del tamaño del beneficio. Al lado, entre qué valores caería un original que fuese una re-ejecución más.

Dos avisos para leerlas. Uno: spread, slippage y estrés combinado sortean costes casi siempre peores que los del original, así que ahí el original queda por encima de la mediana **por construcción** — en el Nikkei, percentil 65–70 en spread y slippage en las diez estrategias —; compara cada tarea consigo misma entre

estrategias, no con 50. Dos: todavía no se sabe qué valores da una estrategia sin ventaja, porque hace falta pasar por las ocho tareas un lote de estrategias de entrada aleatoria (un trabajo de SQX). Hasta entonces son **diagnóstico**: se ven, se pueden leer como hechos, pero no se ofrecen como criterio en la zona Criterios.

Un ejemplo completo

```
$ python3 -m core.assets XAUUSD          # preflight obligatorio antes de nada
$ python3 -m studies.breakage.mcRetest.ingest --project XAUUSD --databank MCR_All --day 2026-
09-18
ingest: 40 runs, 39996 simulations ->
/home/sergioguslw/Desktop/AlgoData/raw/XAUUSD/MCR_All/2026-09-18
  reconciled 30 metrics against SQX, 0 disagreements
  level tables unusable (run cut short): 3 — bar/1.19.29, exits/1.19.29, ohlc/41.5.25
```

Y así se mira lo que ha escrito:

```
from studies.breakage.mcRetest.measure import store
s = store.load_sims(project="XAUUSD", databank="MCR_All", day="2026-09-18")
print(s.groupby("task").NetProfit.median().round(0))
```

```
task
bar         23229.     <- el control: casi no se mueve, que es lo que debe pasar
exits       23766.
mindist     23064.     <- no perturbó nada: los stops están lejos del precio
ohlc        23302.
params      18887.
spread      20969.
slippage    16262.     <- el slippage duele el triple que el spread
stress      11837.     <- las seis cosas a la vez
```

El segundo comando: el informe

La ingesta deja el parquet; el informe lo lee y da el veredicto.

```
python3 -m studies.breakage.mcRetest.report --project XAUUSD --databank MCR_All --day 2026-09-
18
```

Mismos flags que la ingesta (`--project` , `--databank` , `--day` , `--set`), más `--strategy` para una sola estrategia: vale `1.26.46` (como la nombra la ingesta) o `"Strategy 1.26.46"` (como la nombra el databank), y reescribe solo su `estrategias/<nombre>.json` y su página — `verdict.csv` , `mcRetest.json` y lo que sólo se ve entre estrategias no se tocan. Es lo que hace «Run solo esta estrategia» en la ventana. Tarda unos segundos y escribe en `~/Desktop/AlgoData/reports/<proyecto>/<databank>/<fecha>/mcRetest/` (hasta el 25-09, `retest/`):

fichero	qué es
`mcRetest.html` / `.md` / `.json`	el databank: los veredictos y lo que sólo se ve entre estrategias
`estrategias/<nombre>.html` / `.json`	una estrategia: Veredicto, Qué la rompe, Estrés combinado, Las ocho tareas y Tabla de confianza
`verdict.csv`	una fila por estrategia, con su identidad, para filtrar y para `/curate`
`manifest.json`	qué configuración produjo ese veredicto

Lo que sale por pantalla:

```
report: 5 strategies -> /home/sergioguslw/Desktop/AlgoData/reports/XAUUSD/MCR_All/2026-09-18/mcRetest
strategy verdict  composite    binding  stress_net_p5  stress_cvar_dd_pct
 17.9.39    FAIL        2.9 production      -10437.83               28.25
23.16.37    FAIL        7.8 production      -25184.63               45.35
```

En la pestaña «Qué la rompe» cada barra va en verde o rojo según la regla de 1 σ del propio estudio.

`binding` es la subnota más baja: te dice por dónde se rompe, no solo que se rompe. Cinco veredictos posibles: `STRONG`, `ACCEPTABLE`, `MARGINAL`, `FAIL` e **`INCONCLUSIVE`**, que no significa que fallara sino que la batería no dio evidencia para juzgarla.

> **Los umbrales son valores por defecto, no una política.** Cuánto drawdown aguanta la cuenta o qué parte del beneficio debe sobrevivir a la ejecución salen del `config.yaml` y **no están calibrados contra tu operativa**. Si fallan todas, mira primero el umbral. Se cambia sin tocar código: `--set gates.survival_dd_pct=0.35`.

Tocar los umbrales y ver el efecto

El panel del navegador (`explorer.serve`, puerto 8767) **se retiró el 25-09**. Su cajón de ajustes vive ahora en la ventana de escritorio (`ui/`), que corre el mismo `report` con los `--set` que elijas y pinta las mismas cinco pestañas. Desde el terminal es igual de corto:

```
python3 -m studies.breakage.mcRetest.report --project XAUUSD --databank MCR_All --day 2026-09-18 \
    --set gates.exec_keep_frac=0.30
```

El veredicto se recalcula con ese corte sin tocar el `config.yaml`. Es la forma de calibrar los umbrales mirando el efecto en vez de adivinando. Probado: bajar `gates.exec_keep_frac` de 0.70 a 0.30 movió una estrategia de 32,7 a 38,2 y dejó las demás intactas. Cada knob tiene su frase en `studies/breakage/mcRetest/tooltips.py`.

> **Relajar un umbral hasta que una estrategia pase no es calibrar, es elegir la respuesta.** El `--set` sirve para ver cuánto margen hay, no para fabricarlo.

La ventana **no calcula nada propio**: pinta el mismo `estrategias/<nombre>.json` que dibuja la página HTML, así que las dos no pueden discrepar.

Sin caché, a propósito: lo caro ya pasó en la ingesta, así que cada número en pantalla sale de la ejecución que acabas de lanzar y no de una guardada.

Desde el 2026-10-07: qué juzga el paso 14

> **Umbrales PROVISIONALES** (dueño, 2026-10-07): son los de la propuesta, sin calibrar con población nula ni plantada. Pasar aquí significa «no es frágil», no «tiene ventaja». Lote de vecinas y corte sobre la etapa nueva: **primera ejecución real pendiente**.

El paso 14 es **una columna, sin regla** desde el 2026-10-12 (encargo 65): `criteria.yaml` clave `"14"` tiene `rules: []`, así que no juzga ni corta. T2 y T3 se han borrado. Queda **T1**, que entrega hechos (`mcRetest.fragilidad.t1.*`: `ret_med`, `ret_p5`, `f_pos`, `ret_benef`, `beta_share`, `n` en build y oos1, y `rho_vec`) para que el 16.5 los enseñe. Dos hechos nuevos, sin umbral: `percentil_madre` (dónde queda la madre entre sus vecinas en oos1, por exceso neto de deriva, 0 a 100) y `dd_r_p99` (percentil 99 del drawdown máximo de las vecinas sobre build + oos1 juntos, en R: la dispersión que fija k). Salen como columnas de la pestaña «Fragilidad» del informe del paso 14 (`mcRetest.fragilidad.t1.oos1.percentil_madre` y `...t1.ambas.dd_r_p99`); aún no hay camino que haga al 16.5 leerlas de ahí. Sin lote de vecinas no hay hechos.

Dónde se cambian los números del estudio: `studies/breakage/mcRetest/config.yaml › fragility` (ver el capítulo 32 para la tabla completa); los cortes, en `criteria.yaml`.

Qué NO te dice

- **No te dice si la ventaja es real.** Eso se da por supuesto; aquí solo se mide de qué depende.
- **Un nivel de confianza no es un escenario.** El nivel 95 de `NetProfit` y el nivel 95 de `Drawdown` salen de **simulaciones distintas**: cada métrica se ordena por su cuenta. Leídos como pareja describen una corrida que no existió nunca.
- **`MaxLoss` está ordenado al revés.** Su nivel 100 es la peor operación **más suave** de la corrida, no la más dura. Leerlo como cifra de estrés es exactamente al contrario.
- **Solo 30 de las 148 métricas se reconstruyen por simulación.** `SharpeRatio`, `SortinoRatio`, `UlcerIndex`, `RSquared` y `Stability` las calcula SQX sobre la **equity diaria**, que ninguna simulación lleva. Otras 60 —MAE, MFE, exposición, duración— solo existen como los once percentiles.
- **Las tareas 1 a 7 y la 8 no se comparan de tú a tú.** Las siete primeras corrieron solo en muestra; la 8 corrió sobre la muestra completa y su backtest de referencia tiene un 50% más de operaciones. Son backtests distintos.

Si algo falla

`0 retest results, expected one` — ese `.sqx` no lleva un Monte Carlo Retest. Suele ser un databank de Monte Carlo de *operaciones*, que guarda un fichero de órdenes parecido pero no es lo mismo. Comprueba que estás apuntando al databank correcto.

`ran ['RandomizeSpread', 'RandomizeSlippage'], expected ['RandomizeSpread']` — esa tarea corrió con dos métodos activos. Vuelve a correrla en SQX con uno solo, o el estudio no podrá atribuir nada.

`already holds an ingest` — ya existe un export de esa fecha. Usa otra `--day` o borra la carpeta. No sobrescribe a propósito: parquet **añade** en vez de reemplazar, y una ingesta repetida duplicaría todas las filas en silencio.

`REFUSING to write — N reconstructions disagree with SQX` — alguna fórmula reconstruida ya no reproduce lo que SQX calculó. No es un aviso: no escribe nada. Suele significar que alguien tocó `studies/breakage/mcRetest/model/` sin volver a calibrar.